

Raport projektowy: klasyfikacja obrazów pomieszczeń z wykorzystaniem sieci neuronowych

Michał Dyło, Agnieszka Wardak

26 czerwca 2026

Spis treści

1	Wstęp i cel projektu	3
2	Dane i konfiguracja zadania	3
3	Wykorzystane technologie	4
4	Kolejne eksperymenty modelowe	5
4.1	Eksperyment 1: MLP	5
4.2	Eksperyment 2: proste CNN bez augmentacji	6
4.3	Eksperyment 3: proste CNN z augmentacją	6
4.4	Eksperyment 4: ResNet bez augmentacji	6
4.5	Eksperyment 5: ResNet z augmentacją i ensemble	7
5	Hiperparametry i rozdzielczość wejścia	8
5.1	Wpływ hiperparametrów	8
5.2	Wpływ downscale'u	9
6	Wyniki eksperymentów	10
6.1	Porównanie modeli w kolejności zaawansowania	10
6.2	Różnice między eksperymentami i przebieg treningu	11
6.3	Wyniki per klasa	13
6.4	Macierze pomyłek	13
7	Podsumowanie	15

1 Wstęp i cel projektu

Celem projektu było rozwiązanie zadania klasyfikacji obrazów pomieszczeń. Dla pojedynczego zdjęcia RGB model miał przewidzieć jedną z czterech klas: `bed`, `din`, `kitchen` albo `living`. Jest to klasyfikacja wieloklasowa, ale trudniejsza niż rozpoznawanie pojedynczych obiektów, ponieważ klasy są opisywane przez całą scenę. Salon, jadalnia i kuchnia często współwystępują w jednej przestrzeni, a zdjęcia różnią się perspektywą, oświetleniem, stylem wnętrza i liczbą widocznych mebli.

Raport ma strukturę eksperymentalną. Zamiast opisywać tylko finalny model, kolejne rozdziały pokazują, jak zmieniała się jakość po dodaniu następnych elementów:

- MLP jako najprostszy punkt odniesienia,
- proste CNN bez augmentacji,
- proste CNN z augmentacją,
- własny model typu ResNet bez augmentacji,
- ten sam typ modelu ResNet z silną augmentacją i ensemble.

Dodatkowo sprawdzono wpływ hiperparametrów, rozdzielczości wejścia oraz `downscale'u`. Wszystkie sieci były trenowane od zera. Nie użyto transfer learningu ani gotowych wag, więc porównanie dotyczy architektury, danych wejściowych i procedury uczenia, a nie wiedzy przeniesionej z dużego zewnętrznego zbioru.

2 Dane i konfiguracja zadania

Wykorzystano zbiór *House Rooms & Streets Image Dataset* z Kaggle [1]. Dataset zawiera katalogi `house_data` oraz `street_data`; w projekcie użyto wyłącznie katalogu `house_data`, ponieważ dotyczył zdjęć wnętrz. Etykieta była zapisana w nazwie pliku jako prefiks, np. `bed_...jpg`.

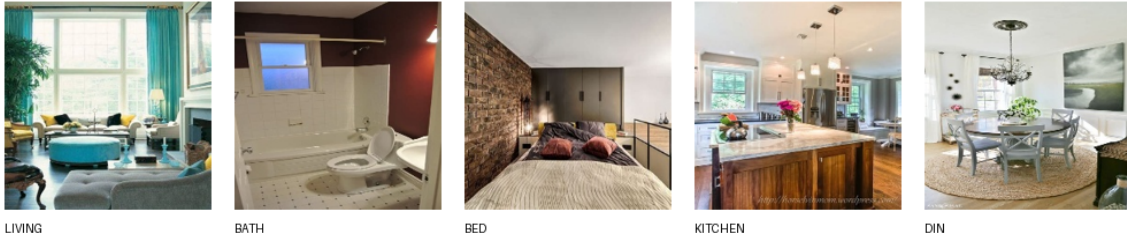
Tabela 1: Liczebność klas w pełnym katalogu `house_data`.

Klasa	Liczba obrazów
<code>bath</code>	605
<code>bed</code>	1248
<code>din</code>	1158
<code>kitchen</code>	965
<code>living</code>	1273
Razem	5249

Klasę `bath` usunięto z finalnego eksperymentu, aby zachować zgodność z wcześniejszą wersją notebooka oraz skupić się na klasach bardziej podobnych wizualnie. Po usunięciu tej klasy pozostały 4644 obrazy. Dane podzielono stratyfikacyjnie na zbiory treningowy, walidacyjny i testowy w proporcji 70%/15%/15%. Ten sam podział był używany dla wszystkich modeli.

Tabela 2: Stratyfikowany podział danych po usunięciu klasy **bath**.

Podzbiór	bed	din	kitchen	living	Razem
Treningowy	873	811	675	891	3250
Walidacyjny	187	174	145	191	697
Testowy	188	173	145	191	697



Rysunek 1: Przykładowe obrazy z katalogu `house_data`. W finalnych eksperymentach pominięto klasę **bath**.

Klasy są dość zbalansowane, ale nie idealnie. Najmniej przykładów ma klasa **kitchen**, a najwięcej **living**. W modelach **RoomResNet** zastosowano wagi klas obliczone na podstawie liczebności zbioru treningowego, natomiast proste modele CNN trenowano bez wag klas, aby utrzymać prostszą procedurę uczenia.

Dla wszystkich modeli obrazy konwertowano do RGB, skalowano do odpowiedniego rozmiaru i normalizowano przy użyciu średniej oraz odchylenia standardowego ImageNet:

$$\mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225).$$

Ta normalizacja nie oznacza użycia transfer learningu; jest tylko standardowym przeskalowaniem kanałów wejściowych.

Jako główne metryki raportowano accuracy oraz macro F1. Macro F1 jest ważne w tym zadaniu, ponieważ traktuje wszystkie klasy równorzędnie i pokazuje, czy model nie poprawia wyniku kosztem jednej trudniejszej klasy. Finalne wyniki pochodzą z noteboka ewaluacyjnego `evaluate_trained_models.ipynb`. Główne warianty uruchomiono dla pięciu seedów: 42, 43, 44, 45 oraz 46.

3 Wykorzystane technologie

Eksperymenty przygotowano jako zestaw skryptów treningowych w Pythonie oraz osobny notebook ewaluacyjny. Najważniejsze technologie pełniły następujące role:

- **Python** – główny język implementacji pipeline’u treningowego, ewaluacji i agregacji wyników.
- **PyTorch** – implementacja architektur MLP, CNN i **RoomResNet**, pętli treningowej, funkcji straty, optymalizatorów, schedulerów, EMA wag, MixUp/CutMix oraz zapisu checkpointów.

- **TorchVision** – skalowanie obrazów i augmentacje, m.in. cropy, odbicia, jitter kolorów, RandAugment oraz RandomErasing.
- **NumPy, pandas i scikit-learn** – przygotowanie podziałów danych, liczenie metryk, macierze pomyłek i agregacji wyników po seedach.
- **Matplotlib, Seaborn i TensorBoard** – wykresy treningu, porównania modeli, macierze pomyłek i monitoring metryk epokowych.
- **Jupyter Notebook** – końcowa ewaluacja wytrenowanych modeli, porównanie seedów, przygotowanie tabel i wykresów użytych w raporcie.
- **CUDA i SLURM** – uruchamianie treningów na klastrze GPU. Skrypty `sbatch` pozwalały odpalać wiele konfiguracji i seedów jako osobne joby.
- **LaTeX i Tectonic** – przygotowanie finalnego raportu PDF, spisu treści, tabel, równań i osadzenie wykresów z ewaluacji.

4 Kolejne eksperymenty modelowe

4.1 Eksperyment 1: MLP

Pierwszym punktem odniesienia był MLP. Model dostaje obraz po spłaszczeniu do jednego wektora pikseli, więc nie ma wbudowanego mechanizmu wykrywania lokalnych wzorców, takich jak krawędzie, narożniki, tekstury czy układ mebli. Ten eksperyment sprawdzał, jak daleko można dojść samymi warstwami gęstymi.

Działanie MLP jest najprostsze ze wszystkich badanych modeli. Obraz RGB 96×96 jest zamieniany na wektor liczb, a kolejne warstwy liniowe wykonują nieliniowe przekształcenia tego wektora. BatchNorm stabilizuje skalę aktywacji, ReLU wprowadza nieliniowość, a Dropout losowo zeruje część neuronów w czasie treningu, żeby ograniczyć zapamiętywanie zbioru treningowego. Ostatnia warstwa zwraca cztery logity, które po funkcji softmax można interpretować jako prawdopodobieństwa klas.

Tabela 3: Architektura modelu MLP.

Element	Opis
Wejście	obraz RGB 96×96 , czyli 27648 cech po spłaszczeniu
Warstwa 1	Linear(27648, 1024) + BatchNorm + ReLU + Dropout(0.50)
Warstwa 2	Linear(1024, 512) + BatchNorm + ReLU + Dropout(0.50)
Warstwa 3	Linear(512, 256) + BatchNorm + ReLU + Dropout(0.375)
Wyjście	Linear(256, 4)

Najlepszy wariant MLP używał obrazów 96×96 , learning rate $5 \cdot 10^{-4}$, weight decay 10^{-4} , dropout 0.50 i label smoothing 0.03. Większy rozmiar wejścia szybko zwiększał liczbę parametrów w pierwszej warstwie, dlatego MLP nie był sensownym miejscem na wysoką rozdzielczość.

4.2 Eksperyment 2: proste CNN bez augmentacji

Drugim krokiem było proste CNN trenowane bez augmentacji. W porównaniu z MLP model konwolucyjny zachowuje informację o lokalnym sąsiedztwie pikseli i współdzieli wagi w różnych miejscach obrazu. Dzięki temu może uczyć się cech wizualnych niezależnie od ich dokładnego położenia.

Mechanika działania CNN polega na budowaniu map cech. Warstwy Conv2D przesuwają małe filtry po obrazie i wykrywają lokalne wzorce, np. krawędzie, kontrasty, tekstury oraz proste fragmenty obiektów. Po konwolucjach stosowany jest MaxPool, który zmniejsza rozdzielczość map cech i zostawia najsilniejsze aktywacje. Dzięki temu kolejne warstwy widzą coraz większy kontekst obrazu, a model staje się mniej wrażliwy na niewielkie przesunięcia mebli lub elementów sceny. Na końcu mapy cech są spłaszczane i przekazywane do klasyfikatora liniowego.

Proste CNN składało się wyłącznie z warstw Conv2D, BatchNorm, ReLU, MaxPool, Dropout oraz Linear. Nie użyto bloków rezydualnych, Squeeze-and-Excitation, MixUp/CutMix ani transfer learningu. Modele bez augmentacji trenowano na deterministycznie przeskalowanych obrazach, bez losowych modyfikacji danych wejściowych. W tej grupie porównano wariant z mocnym downscalem do 64×64 oraz wariant 224×224 .

4.3 Eksperyment 3: proste CNN z augmentacją

Trzeci eksperyment sprawdzał wpływ augmentacji przy tej samej rodzinie prostych CNN. W ramach danej rozdzielczości architektura CNN i CNN+aug była taka sama; różniły się dane widziane w czasie treningu. Celem było oddzielenie wpływu samej architektury od wpływu losowych przekształceń obrazu.

Działanie augmentacji polega na tym, że model nie dostaje w każdej epoce dokładnie tych samych obrazów. Transformacje są nakładane w locie podczas generowania batchy treningowych. Dzięki temu sieć widzi wiele lekko zmienionych wersji tych samych zdjęć i trudniej jej zapamiętać konkretne piksele. Wymusza to uczenie bardziej ogólnych cech pomieszczenia, np. układu przestrzeni, obecności blatów, łóżek, stołów czy kanap, a nie dokładnego kadru jednego pliku.

Dla prostych CNN z augmentacją użyto lekkiego profilu:

- losowe odbicie poziome z prawdopodobieństwem 0.5,
- losowe przemnożenie jasności przez wartość z zakresu 0.8–1.2 z prawdopodobieństwem 0.5,
- losowe przesunięcie cykliczne obrazu do 4 pikseli z prawdopodobieństwem 0.5.

4.4 Eksperyment 4: ResNet bez augmentacji

Następnie sprawdzono mocniejszą architekturę konwolucyjną trenowaną od zera. Model nazwano **RoomResNet**. Jest to własna sieć typu ResNet, a nie gotowy model z biblioteki. Architektura wykorzystuje ideę połączeń rezydualnych [3] oraz modułów Squeeze-and-Excitation [4]; łączy bloki rezydualne, BatchNorm, aktywację SiLU i global average pooling.

W tym modelu obraz przechodzi przez kolejne etapy rezydualne. Każdy etap zwiększa liczbę kanałów, czyli liczbę wykrywanych typów cech, a jednocześnie stopniowo redukuje

rozdzielczość map cech. Początkowe warstwy uczą się prostych wzorców, takich jak krawędzie i tekstury. Głębsze warstwy łączą je w bardziej abstrakcyjne informacje o scenie, np. układ dużych mebli, obecność stołu, łóżka albo fragmentów kuchni. Global average pooling zamienia końcowe mapy cech na jeden wektor opisujący cały obraz, a klasyfikator liniowy przypisuje ten wektor do jednej z czterech klas.

Połączenia rezydualne ułatwiają uczenie głębszej sieci, ponieważ blok nie musi od nowa uczyć całej transformacji. Może nauczyć się tylko poprawki dodawanej do wejścia bloku. Moduł Squeeze-and-Excitation dodatkowo waży kanały cech: wzmacnia te, które są ważne dla danej fotografii, i osłabia mniej istotne.

Pojedynczy blok rezydualny ma postać:

$$\begin{aligned} x &\rightarrow \text{Conv}_{3\times 3} \rightarrow \text{BN} \rightarrow \text{SiLU} \rightarrow \text{Dropout2D} \\ &\rightarrow \text{Conv}_{3\times 3} \rightarrow \text{BN} \rightarrow \text{SE} \rightarrow +\text{shortcut} \rightarrow \text{SiLU}. \end{aligned}$$

Jeżeli liczba kanałów lub rozdzielczość zmienia się między wejściem i wyjściem bloku, shortcut zawiera konwolucję 1×1 ze stride.

Tabela 4: Konfiguracje architektury RoomResNet.

Wariant	Kanały stem	Kanały etapów	Liczba bloków	Parametry
small	32	48, 96, 192, 320	2, 2, 2, 2	ok. 5.08 mln
medium	48	64, 128, 256, 384	2, 2, 3, 2	ok. 9.15 mln
large	64	80, 160, 320, 512	2, 3, 4, 2	ok. 17.63 mln

Wariant bez augmentacji był ważną kontrolą: pokazywał, czy sama większa architektura wystarczy do poprawy generalizacji. Model był oceniany dla wejścia 224×224 .

4.5 Eksperyment 5: ResNet z augmentacją i ensemble

Ostatnim i najbardziej zaawansowanym wariantem był RoomResNet trenowany z silną augmentacją. Użyto tej samej rodziny architektur, ale zmieniono procedurę uczenia i przetwarzania danych.

Ten eksperyment łączył trzy mechanizmy poprawiające generalizację. Pierwszym była silna augmentacja obrazu, która symulowała różne kadry, perspektywy, jasność i drobne zniekształcenia. Drugim były MixUp [6] i CutMix [7], które utrudniały modelowi podejmowanie zbyt pewnych decyzji na podstawie pojedynczych lokalnych fragmentów. Trzecim było TTA w ewaluacji: model oceniał kilka wersji tego samego obrazu, a końcowa predykcja była średnią prawdopodobieństw. W praktyce zmniejsza to wpływ jednego przypadkowego kadrowania na wynik.

W profilu **strong** zastosowano:

- RandomResizedCrop do rozmiaru wejściowego, skala 0.62–1.00, proporcje 0.80–1.25,
- losowe odbicie poziome,
- ColorJitter dla jasności, kontrastu, nasycenia i barwy,
- RandAugment [8] z dwoma operacjami o sile 7,

- losową skalę szarości, autokontrast i zmianę ostrości,
- GaussianBlur,
- RandomAffine oraz RandomPerspective,
- RandomErasing po normalizacji.

Dodatkowo użyto MixUp i CutMix. MixUp tworzy wypukłą kombinację dwóch obrazów i etykiet, a CutMix wkleja fragment jednego obrazu do drugiego i miesza etykiety proporcjonalnie do pola wklejonego fragmentu. W ocenie modeli ResNet+aug zastosowano test-time augmentation (TTA): dla każdego obrazu wykonywano 5 predykcji i uśredniano prawdopodobieństwa.

Na końcu zbudowano ensemble trzech najlepszych modeli ResNet+aug z grid searcha. W tabelach wyników oznaczono je jako A, B i C. Nie są to różne typy architektury: wszystkie trzy warianty używają `RoomResNet large`, silnej augmentacji, EMA wag i TTA. Różnią się wybranymi hiperparametrami uczenia, głównie learning rate, dropoutem w klasyfikatorze i prawdopodobieństwem użycia MixUp/CutMix.

Tabela 5: Definicja wariantów ResNet+aug A/B/C używanych w tabelach wyników.

Wariant	Learning rate	Weight decay	Dropout	p_{mix}
ResNet+aug A	$2 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	0.40	0.45
ResNet+aug B	$3 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	0.40	0.30
ResNet+aug C	$2 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	0.45	0.45

Ensemble nie wymaga dodatkowego treningu. Dla każdego obrazu liczone są prawdopodobieństwa trzech modeli A, B i C, następnie są one uśredniane, a przewidywana klasa to argmax średniego wektora prawdopodobieństw.

5 Hiperparametry i rozdzielczość wejścia

5.1 Wpływ hiperparametrów

Modele trenowano w PyTorch [2]. MLP oraz modele `RoomResNet` trenowano z optymalizatorem AdamW [5]. Proste modele CNN trenowano optymalizatorem Adam, zgodnie z prostszą konfiguracją eksperymentu. Maksymalną liczbę epok ustawiono wysoko, ale trening kończono przez early stopping na podstawie macro F1 na zbiorze walidacyjnym. Dzięki temu modele mogły trenować długo, ale do ewaluacji wybierano najlepszy punkt walidacyjny.

Strojenie hiperparametrów wykonano etapowo. Dla MLP sprawdzano rozmiar wejścia, learning rate, weight decay i dropout. Dla prostych CNN porównano rozdzielczość, wariant architektury, learning rate i dropout. Dla ResNet+aug wykonano grid search po learning rate, weight decay, dropout i prawdopodobieństwie MixUp/CutMix.

Tabela 6: Najważniejsze hiperparametry najlepszego pojedynczego modelu ResNet+aug.

Parametr	Wartość
Architektura	RoomResNet large
Rozmiar wejścia	256×256
Liczba parametrów	17.63 mln
Learning rate	$2 \cdot 10^{-4}$
Weight decay	$5 \cdot 10^{-4}$
Dropout w klasyfikatorze	0.40
Label smoothing	0.10
Scheduler	cosine decay z 12 epokami warmup
EMA wag	0.999
MixUp/CutMix	$\alpha_{\text{mixup}} = 0.2$, $\alpha_{\text{cutmix}} = 1.0$, $p = 0.45$
TTA w ewaluacji	5 predykcji na obraz

Tabela 7: Najlepsze konfiguracje z grid searcha dla prostych CNN. Tabela pokazuje pojedyncze wyniki użyte do wyboru konfiguracji, natomiast finalna ocena po pięciu seedach znajduje się w tabeli 9.

Grupa	Konfiguracja	Accuracy	Macro F1
CNN224+aug, wide	lr 10^{-4} , dropout 0.30	75.61%	75.83%
CNN224+aug, deep	lr 10^{-4} , dropout 0.45	75.47%	75.58%
CNN64+aug, deep	lr 10^{-3} , dropout 0.30	74.61%	74.79%
CNN224, deep	lr 10^{-4} , dropout 0.30	67.00%	67.04%
CNN64, deep	lr $5 \cdot 10^{-4}$, dropout 0.30	62.70%	62.39%

W przykładowym runie dla seeda 42 najlepszy pojedynczy model ResNet+aug osiągnął najlepszy punkt walidacyjny w 206. epoce. W tym punkcie walidacyjne macro F1 wyniosło 0.8681, a accuracy 0.8694. W powtórzeniach po pięciu seedach najlepsze punkty walidacyjne dla wariantów ResNet+aug pojawiały się zwykle po kilkuset epokach, co uzasadnia wysoki limit epok oraz łagodny early stopping.

5.2 Wpływ downscale’u

Osobnym eksperymentem był wpływ rozdzielczości wejścia. Downscale do 64×64 upraszcza uczenie, bo znacząco zmniejsza liczbę cech i koszt obliczeniowy, ale usuwa wiele szczegółów: drobne obiekty, faktury, układ blatów, lamp, krzeseł czy przejść między pomieszczeniami. Dla obrazów wewnątrz te szczegóły mogą być istotne.

Przejsie z 64×64 na 224×224 nie polegało jednak tylko na podaniu większego obrazu do tej samej sieci. Przy większej rozdzielczości mapy cech pozostają dużo większe po kolejnych poolingach. Jeżeli architektura ma za mało etapów redukcji rozdzielczości, część gęsta dostaje bardzo długi wektor cech. To zwiększa liczbę parametrów, utrudnia optymalizację i sprzyja przeuczeniu. Dlatego warianty bez silnego downscale’u wymagały

większej architektury: dodatkowego etapu konwolucyjnego, większej liczby poolingów i klasyfikatora dopasowanego do map cech po redukcji.

Tabela 8: Poprawione warianty prostych CNN użyte w eksperymencie z downscalem. Każdy etap kończy się MaxPool 2×2 . Kolumna FC podaje kolejne rozmiary warstw liniowych.

Wariant	Kanały	Konw.	FC	Parametry
CNN64	32–64–128–256	2–2–2–1	4096–384–192–4	2.23 mln
CNN224 wide	32–64–128–192–256	2–2–2–1–1	12544–256–128–4	4.20 mln
CNN224 deep	32–64–128–192–256	2–2–2–2–2	12544–384–192–4	6.77 mln

Warianty 224×224 mają pięć operacji MaxPool, więc rozdzielczość map cech zmniejsza się z 224 do 7. Dla porównania, przy 64×64 podobna liczba redukcji daje znacznie mniejsze mapy cech. To wyjaśnia, dlaczego model bez downscale’u musiał być większy: zachowuje więcej informacji z obrazu, ale potrzebuje dodatkowych warstw, żeby stopniowo zamienić dużą mapę pikseli na zwarty wektor cech.

6 Wyniki eksperymentów

6.1 Porównanie modeli w kolejności zaawansowania

Tabela 9 pokazuje finalne wyniki na zbiorze testowym po pięciu seedach. W tabeli zachowano kolejność eksperymentów: od MLP, przez proste CNN i CNN+aug, do ResNet oraz ResNet+aug. Warianty ResNet+aug A/B/C odpowiadają konfiguracjom z tabeli 5. Skrót RRN na wykresach oznacza **RoomResNet**.

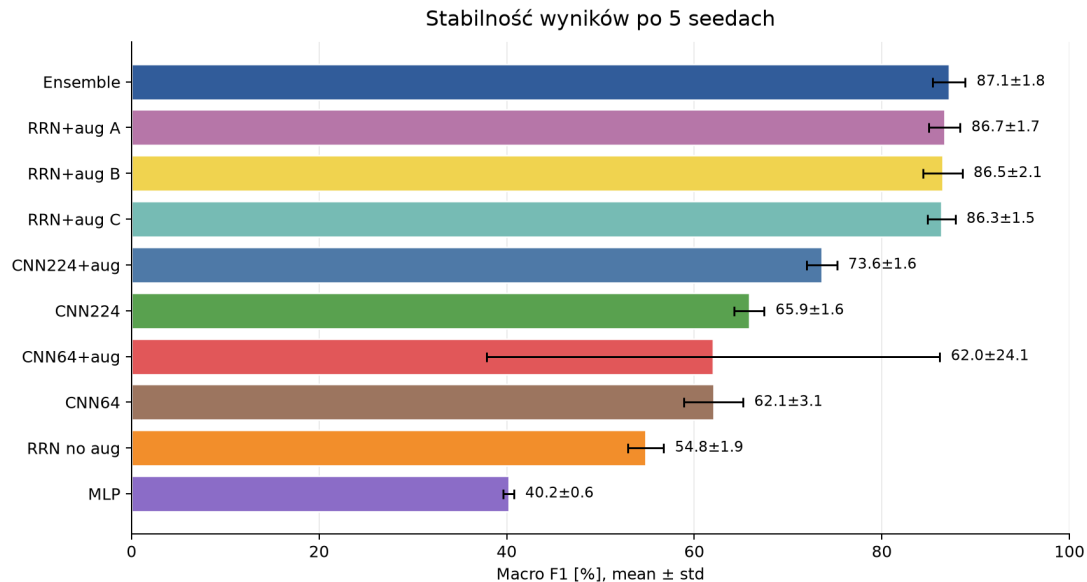
Tabela 9: Porównanie wyników na zbiorze testowym po pięciu seedach. Wartości podano w procentach jako średnia $\pm$ odchylenie standardowe.

Model	Accuracy	Macro recall	Macro F1	TTA
MLP	40.43 $\pm$ 0.49	40.42 $\pm$ 0.45	40.21 $\pm$ 0.56	1
CNN64	62.18 $\pm$ 2.98	61.82 $\pm$ 3.24	62.05 $\pm$ 3.14	1
CNN224	65.82 $\pm$ 1.48	65.96 $\pm$ 1.61	65.86 $\pm$ 1.60	1
CNN64+aug	63.44 $\pm$ 21.02	62.99 $\pm$ 21.47	62.01 $\pm$ 24.15	1
CNN224+aug	73.40 $\pm$ 1.66	73.38 $\pm$ 1.65	73.60 $\pm$ 1.60	1
ResNet no aug	55.01 $\pm$ 1.57	55.24 $\pm$ 1.55	54.82 $\pm$ 1.91	1
ResNet+aug A	86.77 $\pm$ 1.69	86.71 $\pm$ 1.73	86.68 $\pm$ 1.66	5
ResNet+aug B	86.60 $\pm$ 2.14	86.52 $\pm$ 2.15	86.48 $\pm$ 2.09	5
ResNet+aug C	86.48 $\pm$ 1.53	86.40 $\pm$ 1.49	86.35 $\pm$ 1.47	5
Ensemble	87.26$\pm$1.78	87.20$\pm$1.81	87.15$\pm$1.75	5

Wyniki dobrze pokazują sens kolejnych eksperymentów. MLP osiąga tylko 40.21% macro F1, bo ignoruje strukturę przestrzenną obrazu. Proste CNN bez augmentacji poprawia wynik do 62.05–65.86% macro F1. Dodanie lekkiej augmentacji do wariantu 224×224

podnosi wynik do 73.60% macro F1, czyli daje większy zysk niż sama zmiana architektury w obrębie prostego CNN.

Sam ResNet bez augmentacji nie daje dobrego wyniku. To ważna obserwacja: większa architektura bez odpowiedniej regularizacji może przeuczać się do zbioru treningowego i nie musi generalizować lepiej od prostszego CNN. Dopiero połączenie ResNet, silnej augmentacji, MixUp/CutMix, EMA wag i TTA daje skok do około 86–87% macro F1. Ensemble trzech modeli ResNet+aug uzyskuje najwyższą średnią, 87.15% macro F1.



Rysunek 2: Średnie macro F1 oraz odchylenie standardowe po pięciu seedach.

Wariant CNN64+aug ma bardzo duże odchylenie standardowe. Wynika to z jednego nieudanego runa dla seeda 44, w którym macro F1 spadło do 18.92%. Pozostałe seedy tego wariantu były bliżej zakresu 69.93–74.28% macro F1. Wynik zachowano w średniej, ponieważ był częścią zaplanowanego eksperymentu i pokazuje niestabilność tej konfiguracji.

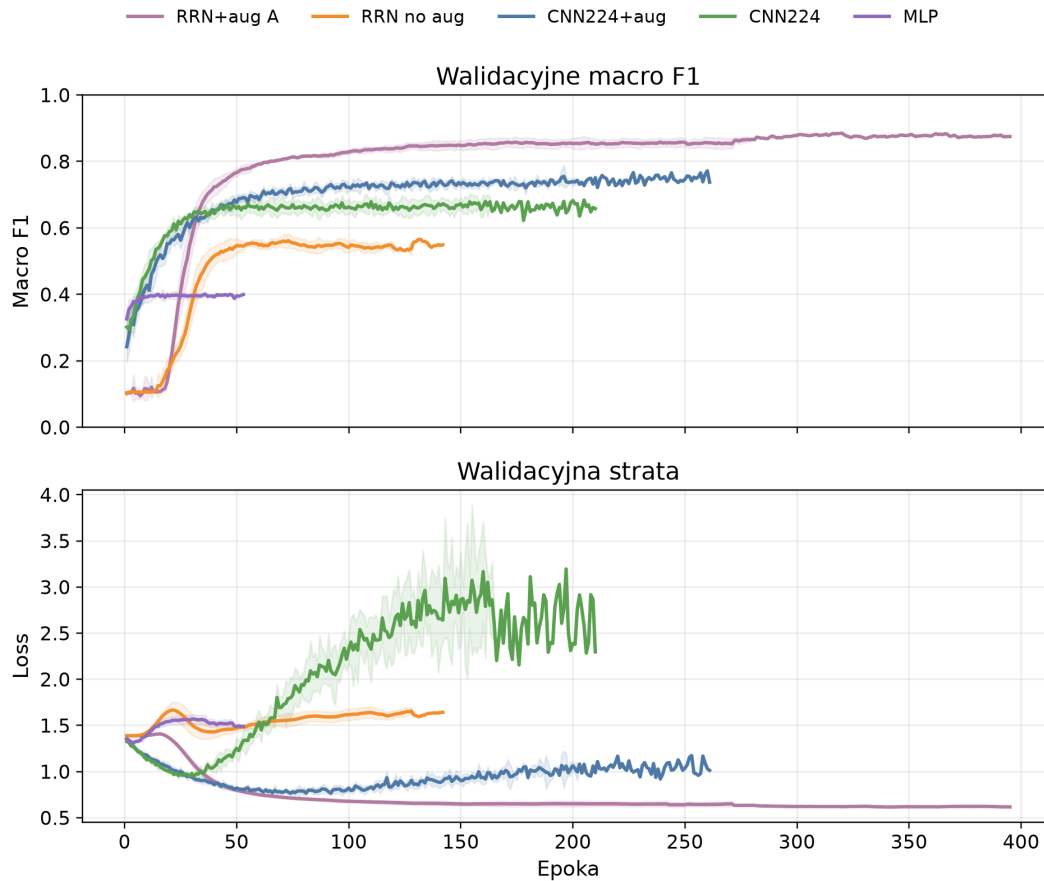
6.2 Różnice między eksperymentami i przebieg treningu

Tabela 10 pokazuje różnice macro F1 liczone parami po tych samych seedach. Przedziały ufności są orientacyjne, ponieważ liczba seedów jest mała ($n = 5$), ale pomagają odróżnić duże efekty od kosmetycznych różnic.

Tabela 10: Różnice macro F1 między wybranymi parami modeli. Wartości podano w punktach procentowych.

Porównanie	Średnia różnica	95% CI
Ensemble vs ResNet+aug A	+0.47	[-0.06, 1.00]
Ensemble vs CNN224+aug	+13.54	[11.81, 15.28]
CNN224+aug vs CNN224	+7.74	[5.40, 10.08]
Ensemble vs ResNet no aug	+32.32	[28.07, 36.58]
CNN224 vs ResNet no aug	+11.04	[7.65, 14.42]

Największe efekty dotyczą augmentacji i kompletnej procedury uczenia. CNN224+aug jest średnio o 7.74 punktu procentowego lepszy od CNN224. Ensemble ResNet+aug jest o 13.54 punktu lepszy od CNN224+aug i o 32.32 punktu lepszy od ResNet bez augmentacji. Różnica między ensemble a najlepszym pojedynczym ResNet+aug jest natomiast mała, więc ensemble traktujemy jako dodatkową stabilizację, a nie główne źródło poprawy.



Rysunek 3: Przebieg walidacyjnego macro F1 i walidacyjnej straty dla wybranych modeli. Linie oznaczają średnią po seedach, a obszary odchylenie standardowe.

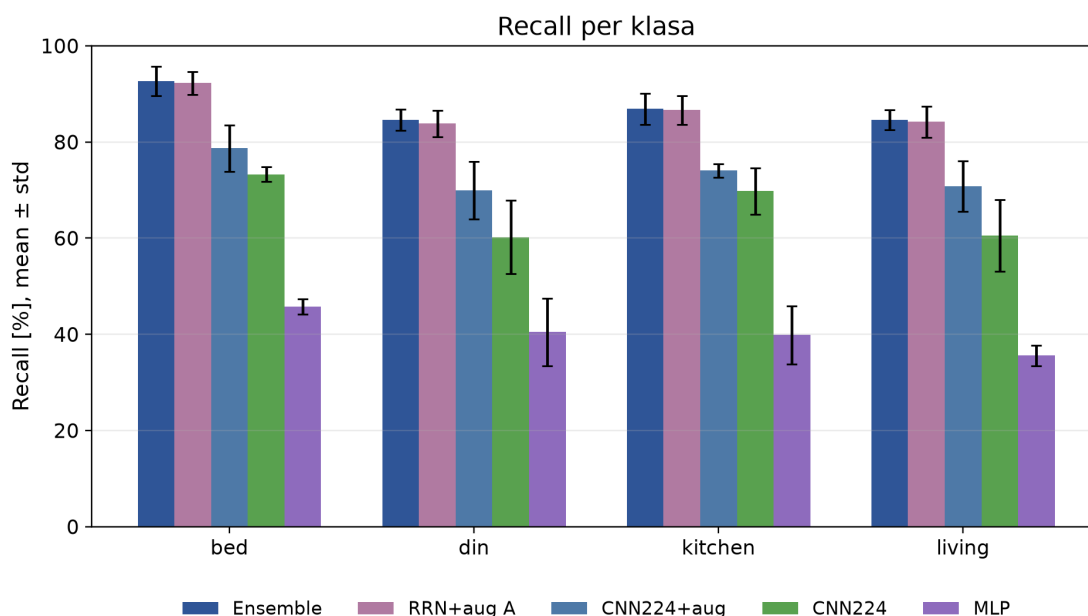
Krzywe treningu potwierdzają obserwacje z tabeli wyników. MLP szybko dochodzi do niskiego plateau. Prosty CNN 224×224 uczy się wyraźnie lepiej, ale jego walidacyjne macro F1 stabilizuje się niżej niż dla modeli z augmentacją. ResNet+aug trenuje dłużej, a najlepsze punkty walidacyjne pojawiają się zwykle po kilkuset epokach.

6.3 Wyniki per klasa

Tabela 11: Metryki per klasa dla finalnego ensemble po pięciu seedach. Wartości podano jako średnia $\pm$ odchylenie standardowe.

Klasa	Precision	Recall	F1	Support
bed	91.30 $\pm$ 0.60	92.66 $\pm$ 3.06	91.96 $\pm$ 1.56	188
din	85.04 $\pm$ 2.62	84.62 $\pm$ 2.22	84.82 $\pm$ 2.30	173
kitchen	85.71 $\pm$ 1.27	86.90 $\pm$ 3.23	86.28 $\pm$ 2.01	145
living	86.50 $\pm$ 3.62	84.61 $\pm$ 2.05	85.52 $\pm$ 2.42	191

Najłatwiejszą klasą dla finalnego modelu pozostaje **bed**, dla której średni recall wynosi 92.66%. Najtrudniejsze są **din** oraz **living**, których recall wynosi około 84.6%. Są to klasy często współwystępujące wizualnie, np. w otwartych przestrzeniach typu salon z jadalnią.



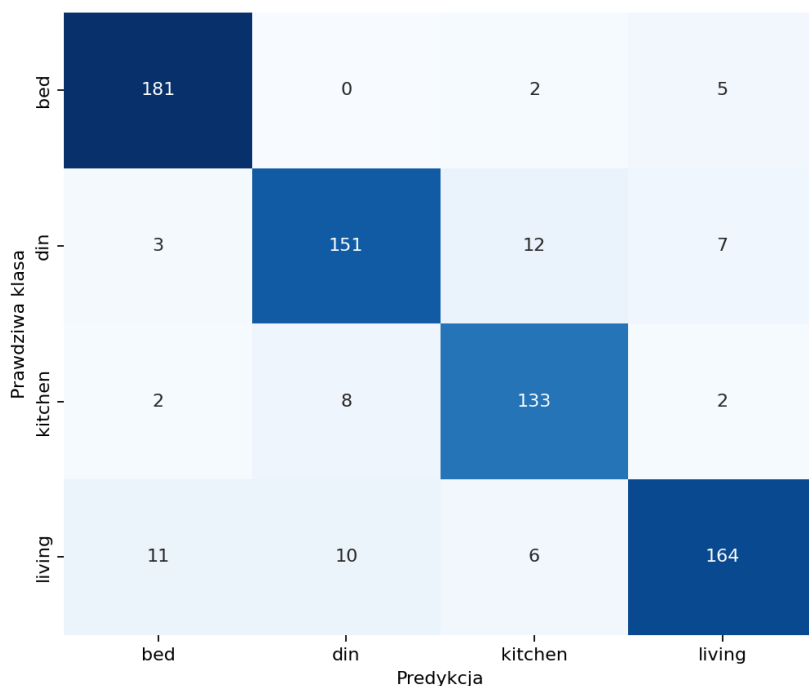
Rysunek 4: Recall per klasa dla wybranych modeli. Słupki pokazują średnią po seedach, a kreski odchylenie standardowe.

6.4 Macierze pomyłek

Macierze pomyłek są zależne od konkretnego seeda, dlatego w tabelach poniżej pokazano reprezentatywny seed 42. W tym seedzie ensemble uzyskało 90.12% macro F1, czyli wynik wyższy niż średnia z pięciu powtórzeń.

Tabela 12: Macierz pomyłek ensemble dla seeda 42. Wiersze oznaczają klasy prawdziwe, kolumny predykcje.

	bed	din	kitchen	living
bed	181	0	2	5
din	3	151	12	7
kitchen	2	8	133	2
living	11	10	6	164



Rysunek 5: Macierz pomyłek ensemble dla seeda 42 zapisana przez notebook ewaluacyjny.

Dla porównania, RoomResNet bez augmentacji w tym samym seedzie znacznie częściej mylił living z bed oraz din. Jego macierz pomyłek przedstawiono w tabeli 13.

Tabela 13: Macierz pomyłek RoomResNet bez augmentacji dla seeda 42.

	bed	din	kitchen	living
bed	136	10	18	24
din	27	70	33	43
kitchen	27	11	88	19
living	71	24	21	75

7 Podsumowanie

Eksperymenty pokazują stopniową drogę od prostego baseline’u do modelu końcowego. MLP jest zbyt słaby dla obrazów pomieszczeń, ponieważ ignoruje strukturę przestrzenną obrazu. Proste CNN wyraźnie poprawia wynik, a przejście z 64×64 na 224×224 pomaga, bo zachowuje więcej informacji wizualnej. Wariant bez downscale’u wymaga jednak większej architektury, ponieważ większe mapy cech muszą zostać stopniowo zredukowane przed klasyfikatorem.

Najważniejsza poprawa pojawia się po dodaniu augmentacji. W prostych CNN augmentacja podnosi wynik wariantu 224×224 z 65.86% do 73.60% macro F1. W przypadku ResNet różnica jest jeszcze większa: ResNet bez augmentacji osiąga 54.82% macro F1, a ResNet+aug około 86–87% macro F1. Oznacza to, że sama mocniejsza architektura nie wystarcza. Dla tego zadania kluczowe są dane widziane w czasie treningu, regularizacja i odporność na zmiany perspektywy, kadru oraz oświetlenia.

Finalny ensemble trzech modeli ResNet+aug osiągnął średnio 87.15% macro F1 po pięciu seedach. Pojedynczy seed 42 dawał wynik ponad 90% macro F1, ale średnia z pięciu seedów jest bardziej wiarygodną oceną jakości. Najtrudniejsze pomyłki dotyczą klas semantycznie podobnych, zwłaszcza `din`, `kitchen` i `living`.

Dalszą poprawę można byłoby uzyskać przez ręczne przejrzanie błędnie sklasyfikowanych obrazów, dodatkową kalibrację ensemble oraz ponowne zbadanie niestabilnych konfiguracji, takich jak CNN64+aug. W projekcie świadomie nie zastosowano transfer learningu; jego użycie prawdopodobnie podniosłoby wynik, ale zmieniłoby charakter porównania.

Literatura

- [1] Mikhail Ma, *House Rooms & Streets Image Dataset*, Kaggle. <https://www.kaggle.com/datasets/mikhailma/house-rooms-streets-image-dataset>
- [2] Paszke, A. et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [3] He, K., Zhang, X., Ren, S., Sun, J., *Deep Residual Learning for Image Recognition*, CVPR, 2016.
- [4] Hu, J., Shen, L., Sun, G., *Squeeze-and-Excitation Networks*, CVPR, 2018.
- [5] Loshchilov, I., Hutter, F., *Decoupled Weight Decay Regularization*, ICLR, 2019.
- [6] Zhang, H. et al., *mixup: Beyond Empirical Risk Minimization*, ICLR, 2018.
- [7] Yun, S. et al., *CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features*, ICCV, 2019.
- [8] Cubuk, E. D. et al., *RandAugment: Practical Automated Data Augmentation with a Reduced Search Space*, CVPR Workshops, 2020.